♡ Sushruth

SMART INDIA HACKATHON · SIH26047

Judge Q&A Handbook

Problem fit, clinical safety, adaptive intake, AYUSH, AI logic, privacy, architecture, demonstration, deployment, and tough questions.

109 prepared answers

Prototype accuracy edition · 14 September 2026
Prepared for presentation and panel discussion

Contents

Important: This handbook distinguishes current functionality from roadmap items. It does not claim live ABDM/ABHA/FHIR, hospital integration, emergency dispatch, offline operation, or production clinical validation.

ABDM / FHIR

OCR / Documents

Technical Architecture

Demo Questions

Business / Deployment

Tough Judge Questions

Good Short Answers To Prepare

Technical Questions Judges May Ask

Improvement and Innovation Questions

Overall Technical Walkthrough

30-Second Closing

Do Not Claim

Sushruth SIH26047 Judge Q&A Handbook

Purpose: Shareable preparation guide for problem fit, clinical safety, AI logic, AYUSH, privacy, architecture, demonstrations, deployment, and tough judging questions.

Accuracy note: This handbook reflects the current prototype as of 14 September 2026. Implemented features are distinguished from proposed production capabilities.

Executive Brief

Sushruth is a platform designed to streamline the patient-to-doctor clinical workflow in India by providing structured, AI-assisted pre-consultation intake. Built on a modern React/Vite and Express/PostgreSQL stack, it reduces administrative friction by summarizing patient symptoms, extracting data from medical reports via AI, and providing clinicians with a verified, structured dashboard to review patient health records before consultation.

One-Minute Pitch

Sushruth transforms the "blank slate" start of doctor consultations into a structured, data-rich interaction. By moving intake to a multilingual, AI-supported digital interface, we reduce the time spent on manual history-taking and document scanning. We provide a safety-first triage layer that alerts users to emergency symptoms, helping patients get to the right care faster while ensuring doctors receive an organized clinical summary they can verify and act upon instantly.

Problem Fit

> **Q: What exact SIH problem statement are you solving?**
>
> A: We are targeting SIH26047’s patient case-taking and pre-consultation requirement. The solution captures a structured patient history before consultation and prepares it for clinician review; we avoid inventing a longer official title where the official listing is not available in the project materials.

> **Q: How is this different from a normal symptom checker?**
>
> A: Unlike standalone checkers, our platform connects structured intake to a clinician review workflow. We do not provide diagnoses; we prepare patient-reported summaries and extracted facts that the physician must verify.

> **Q: Who is the primary user?**
>
> A: The primary users are patients (via the portal) and doctors (via the clinician dashboard).

Q: What pain point does this reduce in OPD or emergency workflows?

A: It reduces the time doctors spend on manual data entry, history-taking, and parsing disorganized patient records, allowing for more time dedicated to diagnosis and treatment.

Workflow

Q: Why is this needed in India specifically?

A: India’s OPD environments are often high-volume and resource-constrained; digitizing the initial intake helps bridge communication gaps, manages language barriers, and organizes critical information that might otherwise be lost in paper records.

Q: Walk us through the full patient journey.

A: A patient completes a multilingual intake questionnaire, uploads medical documents for OCR extraction, and identifies their preferred doctor/clinic. Once submitted, the data moves to a pending state where the doctor reviews, verifies the AI-extracted facts, and confirms the consultation details.

Q: How does patient intake reach the doctor?

A: The intake is submitted via the portal API, stored in a PostgreSQL database, and then retrieved by the doctor's dashboard through an authenticated, role-restricted API call.

Q: What happens if the patient is elderly, low-literacy, or cannot type?

A: We provide support via speech-to-text, large touch targets, multilingual interfaces, and a virtual keyboard to facilitate input without requiring advanced literacy or typing speed.

Q: Can this work in a hospital kiosk?

A: The UI is designed with large elements and touch-friendly controls that are suitable for kiosk environments, though no specific hardware integration or guest-mode workflow is currently implemented.

Q: How much time does it save before consultation?

A: While we have no pilot data, our target is to reduce the initial consultation time by minimizing manual history-gathering and document summarization tasks currently performed by the physician.

Medical Safety

Q: Is your system diagnosing patients?

A: No, the system explicitly disclaims diagnosis and is strictly for triage and administrative summarization.

Q: What happens if the AI gives a wrong suggestion?

A: The system requires a doctor to review and verify every AI-generated summary and extracted fact before any clinical decision is made.

Q: How do you handle red-flag symptoms?

A: We have a deterministic safety screen for six critical symptoms; if any are marked "yes," the system displays an urgent banner instructing the user to call 108 or seek emergency care immediately.

Q: When do you escalate to emergency care?

A: Escalation occurs during safety screening. If a red flag is triggered, the UI shows urgent guidance to call 108 or seek emergency care; it does not notify or dispatch an ambulance. The separate Emergency Alert feature is halted and marked “Feature under development.”

Q: How do you make sure the doctor has final control?

A: The doctor dashboard requires manual review of every section; they can verify, edit, or reject the AI-extracted data, and they must formally confirm the consultation.

AI / Logic

Q: What AI model or logic are you using?

A: We use `gpt-4o-mini` for triage and summary generation, and Claude vision for image/report text extraction.

Q: How does adaptive questioning work?

A: Our engine uses base questions plus category-specific branches that dynamically update based on the patient's selected symptoms and severity levels.

Q: How do you classify urgency?

A: Urgency is determined by deterministic safety checks in the UI and an LLM-based triage logic on the backend that assigns a low to emergency-level status based on the structured input.

Q: How do you extract medicines and diagnoses from reports?

A: The backend uses image extraction via Claude, instructed to pull text accurately without interpreting or diagnosing, which is then presented to the doctor for verification.

Q: How do you handle hallucination?

A: We mitigate this by using structured JSON output, rejecting non-medical input, and forcing human clinical verification of every AI-generated conclusion.

Q: How do you validate clinical summaries?

A: Validation is entirely performed by the attending physician who reviews the summary in their dashboard and marks sections as verified.

AYUSH

Q: Why did you include AYUSH?

A: We include it to capture a holistic health profile, acknowledging the prevalence and importance of traditional medicine systems in Indian patient populations.

Q: What is Dashavidha Pariksha?

A: It is an Ayurvedic framework for assessing patient health through ten specific parameters; we capture data points corresponding to these, such as Prakriti and Vikriti, to provide a structured record.

Q: How does AYUSH intake differ from allopathic intake?

A: The intake captures traditional parameters like constitution (Prakriti) and digestive strength (Agni/Ahara Shakti) alongside standard allopathic clinical history.

Q: Can an AYUSH practitioner use the same dashboard?

A: The review screen can present both standard clinical history and AYUSH parameters. However, the current prototype has a general doctor role and does not yet implement a separately credentialed AYUSH-practitioner role.

Q: How do you avoid mixing unverified advice with clinical triage?

A: AYUSH data is stored as patient-reported, structured metadata, which is clearly separated from clinical triage data and subject to the same physician-verification requirement.

Data / Privacy

Q: What patient data do you store?

A: We store profiles, clinical intakes, health records, medical reports, and appointment data in a secure PostgreSQL database.

Q: How do you get consent?

A: Consent is obtained and timestamped during the pre-consultation intake process, where the patient must review and agree to the sharing of their summary.

Q: Can the patient revoke consent?

A: Currently, consent is provided at the time of submission; we do not have an active, ongoing consent revocation ledger implemented.

Q: Who can see the doctor dashboard?

A: Only authorized users with a `doctor` role in our database can access the dashboard, and access is further filtered to show only patients assigned to them.

Q: How do you protect sensitive medical data?

A: We utilize Clerk for robust authentication and enforce strict server-side ownership checks on all API routes to ensure users only access their own or their assigned patients' data.

Q: How does this align with DPDP / ABDM principles?

A: We prioritize a consent-first design and data minimization by requiring patient input for every record, although we do not currently implement official ABDM standards or external EHR integration.

ABDM / FHIR

Q: Is your ABDM integration live?

A: No, the ABDM integration is not live. There are no references to ABDM within the codebase, and the platform utilizes internal CRUD tables rather than standards-based exchanges.

Q: What is FHIR and why are you using it?

A: FHIR (Fast Healthcare Interoperability Resources) is a standard for exchanging electronic health records. It is not currently implemented in this application, as the system relies on custom data structures and database schemas rather than external health interoperability standards.

Q: How would this connect to ABHA?

A: There is no current connectivity to the Ayushman Bharat Health Account (ABHA) system. Any future connection would require building an integration layer to map the internal user profiles and health records to the ABHA identity and data exchange specifications.

Q: What data becomes part of the FHIR bundle?

A: Currently, no data is bundled into FHIR. The system captures patient-reported intake data, medical history, AYUSH parameters, and uploaded documents, which are stored as internal database records rather than FHIR resources.

Q: What would be needed to make this production-ready?

A: Transitioning to production would require implementing clinical validation, regulatory compliance frameworks, robust EHR/FHIR integration, and a production-grade infrastructure with automated backups and observability. Additionally, the “Feature under development” alerts, such as emergency dispatch, would need full implementation and rigorous end-to-end testing.

OCR / Documents

Q: Can it read handwritten prescriptions?

A: The system uses an AI-based extraction service for image analysis, which is instructed to extract text exactly as written. However, its effectiveness on handwriting is subject to the limitations of the underlying AI model, and all extracted data requires manual human verification.

Q: What happens if OCR is wrong?

A: The system flags all processed documents with a `needsVerification=true` status. Extracted information is presented to the user or doctor for review, and the system relies on human intervention to correct any inaccuracies.

Q: How do you highlight abnormal values?

A: The AI report simplification endpoint is prompted to identify and list abnormal lab values or clinical terms. These are then displayed to the user as identified facts extracted from the report.

Q: Can it create a medical timeline from reports?

A: No, the system does not include a clinical timeline builder. Records are currently ordered by their database creation timestamp, but there is no automated normalization of symptoms or chronological charting of health events.

Q: How do doctors verify extracted information?

A: Doctors use a dedicated review interface where they can see the patient’s HPI, history, AYUSH fields, and extracted document facts. They are required to verify each section individually, enter notes, and manually confirm the consultation.

Technical Architecture

Q: What is your tech stack?

A: The stack consists of a React 19/Vite frontend with Tailwind CSS/shadcn/ui, an Express 5/Node.js backend, and a PostgreSQL database managed via Drizzle ORM. Authentication is handled by Clerk, and OpenAI/Anthropic models are used for AI triage and extraction.

Q: Why Flask and SQLite?

A: The current stack does not use Flask or SQLite. It is built on a TypeScript-based Node.js backend using PostgreSQL, chosen for its scalability and compatibility with the Drizzle ORM ecosystem.

Q: How would you scale this beyond a demo?

A: To scale beyond the current demo, the application would require moving from the current Replit-based hosting to a robust cloud infrastructure. Necessary improvements include implementing read replicas, caching layers, message queues for background processing, and comprehensive observability and monitoring systems.

Q: What APIs have you built?

A: We have built a comprehensive set of RESTful APIs, including endpoints for triage, medical reports, appointment scheduling, health records, medication tracking, care circle management, and pre-consultation intake workflows. All routes are prefixed with `/api` and secured with Clerk authentication.

Q: What tables are in your database?

A: Our database schema includes core tables for `health_records`, `appointments`, and `doctors`, as well as specific tables for `triage_sessions`, `medical_reports`, `medications`, and `care_circle` management. Additionally, there are tables for `child_profiles` and `pre_consultation_submissions`.

Q: How would you deploy this in a hospital?

A: Deploying in a hospital would require integrating with existing EHR/HIS systems through established interoperability standards like FHIR and HL7. It would also require strict adherence to local clinical governance, onsite infrastructure setup, and a transition from patient-reported data to verified clinical workflows.

Demo Questions

Q: Show a high-risk case.

A: If a patient selects red-flag symptoms such as severe chest pain or syncope during the intake, the system flags the status as "Urgent" and displays an immediate, assertive banner advising the patient to contact 108 or proceed to the nearest emergency room.

Q: Show a normal case.

A: A normal case involves the patient completing the multi-stage, multilingual intake process, which captures their chief complaint, HPI, and medical history. Upon submission, the record enters a "pending" status, creating a structured, patient-reported summary available for doctor review.

Q: Show AYUSH mode.

A: The intake captures 12 Dashavidha-style parameters, including *Prakriti*, *Vikriti*, and *Sattva*. These are stored in the database and a selection of these fields, such as constitution, sleep, and diet, are rendered in the clinician's review dashboard.

Q: Show doctor dashboard.

A: The doctor dashboard displays a list of pending pre-consultation submissions filtered by the assigned doctor's ID. Clinicians can view the patient-reported HPI, medical history, AYUSH data, and extracted facts, marking each section as verified or adding notes before confirming the consultation.

Q: Show consent revoke.

A: Consent is currently captured within the initial pre-consultation intake as an accepted flag and timestamp. There is no separate system-wide consent ledger or active revoke workflow currently implemented in the database schema.

Q: Show FHIR export.

A: The system does not currently support FHIR export. All health records are stored in internal, application-owned database tables, and there is no integration with ABDM or external standards-based systems.

Q: Show document timeline from a report.

A: The system displays documents and records based on their database `createdAt` timestamps. It does not currently support an automated clinical symptom timeline builder or longitudinal duration normalization.

Q: Show what happens for chest pain.

A: The intake engine classifies chest pain as a cardiovascular complaint and asks about radiation, character, breathlessness, exertion, sweating, and dizziness. Severe red-flag responses trigger urgent call-108/seek-emergency-care guidance, not an automated emergency alert or dispatch.

Business / Deployment

Q: Who will pay for this?

A: We propose a model based on clinical efficiency gains for providers rather than an advertisement-driven model. Specific pricing structures and buyer profiles are currently being defined as part of our future roadmap.

Q: How will hospitals adopt it?

A: Hospitals would adopt the platform by integrating the pre-consultation portal into their existing intake workflows. Currently, the system is a standalone portal, and we intend to develop APIs to facilitate future interoperability with hospital information systems.

Q: Can PHCs or rural clinics use it?

A: Yes, the platform is designed for accessibility with large touch targets, multilingual virtual keyboards, and voice-to-text capabilities, making it suitable for deployment in rural kiosk-style settings. However, it requires a stable internet connection and is not currently optimized for offline clinical operations.

Q: Does it need internet?

A: Yes, the system requires an active internet connection. It relies on server-side processing for AI triage, report extraction, and database synchronization, as there is no offline-mode or local sync mechanism.

Q: What hardware is needed for a kiosk?

A: The platform is web-based and can run on any standard tablet or desktop computer equipped with a modern browser. It requires a camera for document capture and a microphone for the voice-to-text intake features.

Q: What is your pilot plan?

A: We are currently focused on refining the intake and clinician review workflow. Our proposed pilot targets involve deploying the platform in outpatient settings to measure reductions in patient intake time and improvements in the completeness of pre-consultation summaries.

Tough Judge Questions

Q: Why should doctors trust this?

A: 1 We are not replacing doctors; we are preparing structured, consented pre-consultation summaries. 4 The doctor remains the final authority and can verify, edit, or reject AI output. 5 The strongest value is reducing intake time and improving completeness before consultation.

Q: What if a patient enters fake or incomplete information?

A: All data is explicitly labeled as "patient-reported" and "pending verification." The doctor review workflow requires the clinician to verify the intake information, ensuring that incomplete or inaccurate data is flagged or corrected before it is integrated into the clinical process.

Q: What if the patient speaks a dialect?

A: The platform supports multiple Indian languages and browser speech input where supported. Dialects and accents may reduce recognition quality, so patients can review captured text and switch to tap or typed input; there is no implemented confidence-threshold guarantee.

Q: What if the system misses an emergency?

A: 2 Red flags are treated as alerts, not diagnoses. The system uses deterministic checks for six critical symptoms; however, it is a screening tool, not a diagnostic one. The clinician remains the final authority for clinical assessment, and our disclaimers emphasize that the system does not replace urgent care or emergency medical services.

Q: How do you handle liability?

A: We explicitly disclaim diagnosis and provide warnings that the platform is for screening purposes only. 2 Red flags are treated as alerts, not diagnoses. All generated outputs are presented to the clinician for review, ensuring human oversight is maintained throughout the process.

Q: How will you clinically validate this?

A: We plan staged validation with clinicians: retrospective case review, red-flag sensitivity testing, extraction accuracy measurement, usability studies, and a supervised OPD pilot. We would compare completeness and intake time against manual intake and track every correction; no validation results are claimed yet.

Q: What is your moat compared to existing EMR or health apps?

A: Our moat lies in our India-first, multilingual, and consent-first architecture designed specifically for resource-constrained environments. By focusing on structured pre-consultation summaries, we solve the high-friction intake process that traditional EMRs fail to address effectively.

Good Short Answers To Prepare

1. We are not replacing doctors; we are preparing structured, consented pre-consultation summaries.

2. Red flags are treated as alerts, not diagnoses.

3. ABDM, ABHA, and FHIR are future interoperability work; they are not implemented or claimed as live in the current prototype.

4. The doctor remains the final authority and can verify, edit, or reject AI output.

5. The strongest value is reducing intake time and improving completeness before consultation.

Technical Questions Judges May Ask

Q: 1. What is the overall architecture of this website?

A: The application is a TypeScript monorepo using pnpm workspaces. It features a React 19/Vite frontend with Tailwind CSS and shadcn/ui, and an Express 5/Node backend. Data persistence is managed via PostgreSQL with Drizzle ORM, and API contracts are handled through OpenAPI and Orval-generated hooks.

Q: 2. What happens when a patient enters symptoms?

A: Symptoms are captured through an adaptive intake form that serializes responses into a structured JSON object. The system uses a clinical question engine to add conditional follow-up questions based on the complaint category. This data is then sent to an LLM for triage, which categorizes urgency and provides care suggestions.

Q: 3. What happens when a patient completes the clinical intake form?

A: Upon completion, the intake data is stored in the `pre_consultation_submissions` table with a `pending` status. The submission includes patient-reported history, AYUSH parameters, and symptoms, which are then queued for an assigned doctor to review via the clinician portal.

Q: 4. How does the app decide urgency: low, moderate, or high?

A: The frontend performs a deterministic safety screen for six critical red flags. Separately, the backend utilizes an LLM-driven triage process that evaluates the structured symptom input to classify the patient's urgency and provide an appropriate care recommendation.

Q: 5. How does red-flag escalation work?

A: If a patient triggers a red-flag condition during the safety screen, the UI immediately updates to an urgent status and displays an assertive banner advising the user to contact emergency services like 108 or proceed to the nearest ER. This is a UI-side safety mechanism; automated emergency dispatch integration is not currently implemented.

Q: 6. What happens internally when the patient enters chest pain or breathlessness?

A: The system identifies these keywords as cardiovascular complaints, triggering specific follow-up questions regarding radiation, exertional worsening, and associated symptoms like sweating or dizziness. These inputs are flagged for clinical review and categorized by the LLM triage service as higher priority.

Q: 7. How does the doctor dashboard get patient data?

A: The dashboard fetches data via a backend route that requires a validated `doctor` role within the `portal_profiles` table. The API enforces ownership by filtering submissions based on the `doctorUserId` assigned to the specific pre-consultation record.

Q: 8. How are patient records stored?

A: Records are stored in a PostgreSQL database using Drizzle ORM. Tables are organized by functional domain, including `health_records`, `triage_sessions`, `medical_reports`, and specific portal-related tables for pre-consultations and profiles.

Q: 9. What database tables did you create?

A: We created a comprehensive schema including core health tables like `health_records` and `appointments`, specialized tables for `triage_sessions` and `medical_reports`, and dedicated modules for `care_circle` management, `child_health` tracking, and `portal` submission tracking.

Q: 10. How does the document timeline work?

A: Documents and clinical history items are ordered and retrieved chronologically based on their database `createdAt` timestamps. The current implementation focuses on retrieving records for display rather than a dynamic, trend-based clinical timeline builder.

Q: 11. How do you extract medicines, diagnoses, and abnormal values from reports?

A: The system sends uploaded images to an Anthropic Claude vision model. It is explicitly prompted to extract specific text facts, such as medications and lab values, while ignoring diagnostic interpretation. These extracted facts are then presented to the user for verification.

Q: 12. How does the physician summary combine intake, triage, reports, and history?

A: The clinician UI aggregates patient-reported HPI, medical history, AYUSH fields, and extracted document facts into one review view. The doctor can add notes and mark the submission reviewed or incomplete; reviewed submissions become read-only in the current interface.

Q: 13. How does consent affect access to patient data?

A: Consent is captured as part of the pre-consultation JSON intake. The system ensures that summaries are only surfaced to the destination clinic once the patient has reviewed and explicitly confirmed the selection.

Q: 14. What is the FHIR export and how is it generated?

A: The system does not currently implement FHIR export. Data is managed through internal CRUD tables tailored to our specific application workflow.

Q: 15. Is ABDM integration real or simulated?

A: ABDM integration is not implemented. The project currently relies on internal application-specific database structures.

Q: 16. What APIs are exposed by the backend?

A: The backend exposes RESTful APIs for triage, medical reports, appointment booking, health record management, portal submissions, and Care Circle member management, all protected by Clerk authentication.

Q: 17. Why did you use Flask and SQLite?

A: The current stack does not use Flask and SQLite; it is built on Express/Node.js and PostgreSQL to provide a robust, scalable architecture suitable for handling JSON payloads and relational data.

Q: 18. How would you scale this from local demo to production?

A: The current stateless Express architecture is designed for horizontal scaling. Production migration would involve deploying the application to a managed cloud environment, implementing read replicas for the PostgreSQL database, and setting up automated CI/CD and observability pipelines.

Q: 19. How would you add real OCR later?

A: We currently use an LLM-based vision approach, which can be refined by integrating specialized medical OCR engines or fine-tuned vision models for higher accuracy in recognizing handwritten medical scripts.

Q: 20. How would you add real voice input later?

A: Voice input is currently handled via the Web Speech API with language-specific constraints. We could improve this by integrating high-fidelity ASR (Automatic Speech Recognition) services specialized for Indian regional languages and accents.

Q: 21. How would you secure doctor login in production?

A: We use Clerk v6 for authentication, which provides robust role-based access control. In production, we would enforce strictly scoped API keys and hardware-backed multi-factor authentication for clinician accounts.

Q: 22. How do you prevent AI output from being treated as final diagnosis?

A: We implement strict system prompts that explicitly forbid the AI from making diagnoses. Furthermore, all AI-generated content is labeled as requiring verification, and the final decision-making process is explicitly gated by a human clinician.

Q: 23. How does AYUSH mode store separate medical information?

A: AYUSH parameters like `prakriti` and `vikriti` are stored as part of the patient's intake data schema. The UI captures these as structured fields, which are then passed through the system to be summarized in the doctor's review dashboard.

Q: 24. How is the patient side connected to the doctor side?

A: The connection is facilitated via the `pre_consultation` submission flow, where a patient submits a request that is stored in the database. A doctor with the appropriate role can then retrieve and review these specific submissions via the portal API.

Q: 25. What new technical features did you add after reading the SIH gap document?

A: We implemented the structured clinical triage engine, a multi-stage pre-consultation intake workflow, AI-assisted image extraction for medical documents, and a role-based portal for clinician review to bridge the gap between patient data and clinical insight.

Improvement and Innovation Questions

Q: What improvements can be made?

A: Add a complete correction and audit trail for AI-extracted facts, production consent revocation, specialized medical OCR, stronger multilingual speech recognition, and clinician-configurable intake templates.

Q: What additional components or features can be added?

A: Future components could include FHIR/ABDM interoperability, hospital-system connectors, offline-capable kiosk synchronization, and notifications to consented caregivers. These are roadmap items, not current capabilities.

Q: How can we make the solution more innovative?

A: Add explainable question selection, evidence-linked summaries where every statement points to its patient answer or source document, and privacy-preserving evaluation across hospitals. Innovation should improve safety and workflow quality rather than automate diagnosis.

Q: How can we improve practical applicability?

A: Add a supervised kiosk mode, assisted-intake workflow, low-bandwidth design, local-language audio prompts, hospital-configurable forms, and a clear handoff into existing registration and clinician queues.

Q: How can the solution be scaled?

A: Keep APIs stateless, scale web and API services horizontally, move long AI jobs to queues, use managed PostgreSQL with backups and read scaling, cache safe read-heavy data, and add monitoring and audit logs.

Q: What differentiates our solution from existing solutions?

A: Sushruth focuses on structured, multilingual, consented pre-consultation preparation rather than diagnosis or generic record storage. It combines adaptive questions, red-flag screening, document extraction, AYUSH fields, and physician review.

Q: What additional features can increase impact?

A: Specialized PDF/report extraction, longitudinal trend views, medication reconciliation, supervised rural-kiosk mode, consent receipts, hospital integration, and clinically evaluated language models could increase impact.

Q: How does the solution align with the problem statement?

A: It directly addresses patient case-taking by capturing comprehensive information before consultation and presenting it in a structured form for clinician review.

Q: What are the differentiating factors?

A: India-focused multilingual access, adaptive clinical intake, explicit red-flag guidance, AYUSH-compatible structured fields, document extraction with verification, and a doctor-in-the-loop handoff.

Q: What is novel and innovative?

A: The novelty is the combination of adaptive pre-consultation intake, structured AYUSH assessment, report extraction, accessibility, and clinician verification rather than any single model.

Q: What unique features are implemented or proposed?

A: Implemented features include complaint-specific adaptive questions, at least ten relevant follow-ups, twelve AYUSH assessment fields, voice/text/tap input, triage, report analysis, and doctor review. Interoperability, offline kiosk mode, and production clinical validation are proposed.

Q: Explain implementation details.

A: The TypeScript frontend uses React and Vite. It calls an Express REST API defined by OpenAPI, with Orval-generated React Query clients and Zod validation; PostgreSQL uses Drizzle ORM, authentication uses Clerk, and AI calls run server-side.

Q: Explain design and architecture.

A: The app separates patient and clinician experiences while sharing typed API contracts and one relational data layer. Authentication and ownership checks are server-enforced, and AI outputs remain reviewable records rather than autonomous clinical actions.

> **Q: Why is it feasible and practical?**
>
> A: It runs in a standard browser and uses common web technologies, so a pilot needs ordinary tablets or desktops rather than specialized hardware. Production work centers on clinical governance, validation, integration, and operations.

> **Q: What is the future scope and scalability?**
>
> A: Future work includes production consent management, FHIR/ABDM integration, specialized OCR and speech services, offline-assisted kiosks, stronger auditing, hospital workflow integration, and controlled multi-site evaluation.

Overall Technical Walkthrough

The website functions as a secure, role-based platform that facilitates communication between patients and clinicians. The process begins with the patient performing a multilingual, adaptive symptom intake. This stage uses a client-side engine to capture clinical history, demographics, and AYUSH-specific parameters, storing the data as a pending `pre_consultation` submission in a PostgreSQL database via a secure Express API.

During intake, the system provides immediate safety checks for emergency red flags. If no emergency is detected, the patient can upload medical reports. These are processed on the server using an LLM to extract text facts—such as medications and lab results—which are then marked for human verification. This ensures that the system provides information rather than unauthorized medical interpretations.

Once the intake is submitted, the information moves to the doctor’s portal. A clinician with appropriate authorization can view the structured summary, which aggregates the patient's HPI, uploaded report facts, and reported symptoms. The doctor reviews each section, verifies the patient-provided details, and can then confirm or request further information. The system ensures data integrity and access control by enforcing strict user-ownership checks at every API endpoint, effectively separating patient and clinician workflows while maintaining a shared source of truth in the database.

30-Second Closing

Sushruth is an AI-assisted clinical preparation platform built for India’s unique healthcare landscape. We provide an intuitive, multilingual interface that structures patient intake and digitizes medical records without crossing the line into automated diagnosis. By keeping a human physician at the center of the review process, we ensure clinical safety while significantly reducing the administrative burden on doctors and improving the quality of patient-provider interactions.

Do Not Claim

We do not claim support for live ABDM, ABHA, FHIR integration, hospital EHR interoperability, automated emergency dispatch services, or production clinical validation. We do not claim an offline-first mode or any proprietary measured time savings. All mentions of emergency alerts are currently in development.